# Step 1: Import libraries

In [1]:
```
import numpy as np
```

# Step 2: Activation function and its derivative

In [2]:
```
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    # x is expected to already be sigmoid(output)
    return x * (1 - x)
```

# Step 3: Define the RNN class (forward pass + BPTT)

In [3]:
```
class SimpleRNN:

    def __init__(self, input_size, hidden_size, output_size, learning_rate=0.01)

        # Initialize network parameters
        self.hidden_size = hidden_size
        self.learning_rate = learning_rate

        # Input -> Hidden weights
        self.W_xh = np.random.randn(input_size, hidden_size) * 0.01
        self.b_h = np.zeros((1, hidden_size))

        # Hidden -> Hidden recurrent weights
        self.W_hh = np.random.randn(hidden_size, hidden_size) * 0.01

        # Hidden -> Output weights
        self.W_ho = np.random.randn(hidden_size, output_size) * 0.01
        self.b_o = np.zeros((1, output_size))

    # Forward Pass
    def forward(self, inputs):

        self.hidden_states = {}
        self.outputs = {}

        # Initial hidden state
        self.h_prev = np.zeros((1, self.hidden_size))

        # Process each time step
        for t, x in enumerate(inputs):
            x = x.reshape(1, -1)
            self.h_prev = sigmoid(np.dot(x, self.W_xh) + np.dot(self.h_prev, sel
            self.hidden_states[t] = self.h_prev
            output = np.dot(self.h_prev, self.W_ho) + self.b_o
            self.outputs[t] = output

        return self.outputs[len(inputs) - 1]
```

```
    # Backward Pass - Backpropagation Through Time (BPTT)
    def backward(self, inputs, targets):
        dW_xh = np.zeros_like(self.W_xh)
        db_h = np.zeros_like(self.b_h)

        dW_hh = np.zeros_like(self.W_hh)
        dW_ho = np.zeros_like(self.W_ho)
        db_o = np.zeros_like(self.b_o)

        dh_next = np.zeros_like(self.h_prev)

        for t in reversed(range(len(inputs))):
            x = inputs[t].reshape(1, -1)
            h_t = self.hidden_states[t]
            if t > 0:
                h_prev_t = self.hidden_states[t - 1]
            else:
                h_prev_t = np.zeros_like(h_t)

            d_output = (self.outputs[t] - targets[t].reshape(1, -1))
            dW_ho += np.dot(h_t.T, d_output)
            db_o += d_output

            dh = np.dot(d_output, self.W_ho.T) + dh_next
            dh_raw = dh * sigmoid_derivative(h_t)

            dW_xh += np.dot(x.T, dh_raw)
            db_h += dh_raw
            dW_hh += np.dot(h_prev_t.T, dh_raw)

            dh_next = np.dot(dh_raw, self.W_hh.T)

        # Update weights and biases using gradient descent
        self.W_xh -= self.learning_rate * dW_xh
        self.b_h -= self.learning_rate * db_h

        self.W_hh -= self.learning_rate * dW_hh

        self.W_ho -= self.learning_rate * dW_ho
        self.b_o -= self.learning_rate * db_o
```

# Step 4: Create the training data

In [4]:

```
# Dummy input sequence
input_sequence = np.array([
    [0.1],
    [0.2],
    [0.3],
    [0.4],
    [0.5]
])

# Corresponding target sequence
target_sequence = np.array([
    [0.2],
    [0.4],
    [0.6],
```

```
    [0.8],
    [1.0]
])

print("Input shape :", input_sequence.shape)
print("Target shape:", target_sequence.shape)
```

```
Input shape : (5, 1)
Target shape: (5, 1)
```

## Step 5: Initialize the RNN

In [5]:
```
input_size = 1
hidden_size = 5
output_size = 1

rnn = SimpleRNN(input_size, hidden_size, output_size, learning_rate=0.05)
print("RNN initialized.")
```

```
RNN initialized.
```

## Step 6: Train the RNN

In [6]:
```
epochs = 1000

print("Training Simple RNN...")

for epoch in range(epochs):
    predicted_output = rnn.forward(input_sequence)
    loss = np.mean((predicted_output - target_sequence[-1]) ** 2)
    rnn.backward(input_sequence, target_sequence)

    if (epoch + 1) % 100 == 0:
        print(f"Epoch {epoch + 1}, Loss: {loss:.4f}")
```

```
Training Simple RNN...
Epoch 100, Loss: 0.1543
Epoch 200, Loss: 0.1449
Epoch 300, Loss: 0.1270
Epoch 400, Loss: 0.0989
Epoch 500, Loss: 0.0710
Epoch 600, Loss: 0.0528
Epoch 700, Loss: 0.0430
Epoch 800, Loss: 0.0376
Epoch 900, Loss: 0.0345
Epoch 1000, Loss: 0.0323
```

## Step 7: Test the trained RNN

In [7]:
```
print("\nTesting the trained RNN:")

test_input_sequence = np.array([
    [0.6],
    [0.7],
    [0.8],
    [0.9],
    [1.0]
```

```
])

final_prediction = rnn.forward(test_input_sequence)

print("Input Sequence:", test_input_sequence.flatten())
print("Predicted Final Output:", final_prediction.flatten())
```

```
Testing the trained RNN:
Input Sequence: [0.6 0.7 0.8 0.9 1. ]
Predicted Final Output: [0.89967196]
```

In [ ]:

In [ ]: